Python AI / ML v 1.0

Aria

AI Document Intelligence Platform

Ask questions across any document. Get cited answers.

AUTHOR	STATUS	VERSION	TIMELINE	DATE
Vaibhav Tiwari	Draft - Approved for Development	1.0	3 Weeks to MVP	April 2026

Stack: FastAPI . LangChain . ChromaDB . Next.js . Gemini 2.0 . PostgreSQL . Cloudflare R2

PRODUCT REQUIREMENTS

Table of Contents

01 Problem Statement

Knowledge workers spend an average of **2.5 hours per day** searching for information across documents, reports, and research papers. The core problem is not the lack of information — it is the **inability to query it naturally.**

Existing tools force users to manually read documents, ctrl+F for keywords, and synthesize across multiple files. There is no way to ask:

> *"What did the Q3 report say about churn compared to what this research paper recommends?"*

Aria solves this by turning any collection of documents into a conversational knowledge base — powered by a RAG (Retrieval-Augmented Generation) pipeline with full source citations. Every answer is grounded in your documents with exact references.

Manual Reading	Cross-Reference	No Citation Trail
Users must read every document in full — no ability to query specific answers from hundreds of pages.	Synthesizing insights across 10+ files requires manual notes, tabs, and hours of effort.	Even when answers are found, tracking source document and page number is error-prone.

02 Target Users & Personas

RS — Riya Sharma
Research Analyst, FinTech

Pain Point: Reads 20+ PDFs per week — annual reports, market research, competitor filings. Synthesizing across them takes hours. Wants to ask cross-document questions and get cited answers instantly.

Key Needs: **Cross-document synthesis . Fast retrieval . Citation trail**

AM — Arjun Mehta
CS Student / Developer

Pain Point: Uploads documentation, textbooks, and lecture notes. Needs to query across all of them. Stack Overflow and Ctrl+F are inefficient for complex technical questions.

Key Needs: **Natural language queries . Multi-file workspaces . Technical accuracy**

PK

Priya Krishnan
Startup Founder

Pain Point Has investor docs, contracts, pitch decks, and competitor analysis. Needs to quickly extract specific information without reading everything every time.

Key Needs **Quick extraction . Export answers . Secure storage**

03 Product Vision & Goals

Aria is the interface between humans and their documents. Anyone should be able to upload any file, ask any question, and get a verified, cited answer — in under 3 seconds.

KPI	Target	Description
Answer Latency (p95)	**< 3 seconds**	From query submission to first streamed token
Citation Accuracy	**95%+**	Source match rate — verified against ground truth
Upload Size Limit	**50 MB**	Per file in v1; scalable to 200MB in v2
Supported Formats	**5+ types**	PDF, CSV, XLSX, DOCX, TXT, PPTX
Processing Speed	**< 90 sec**	100-page PDF fully indexed and queryable

04 Scope — In vs Out

Feature	V1 Scope	Rationale
PDF upload + parsing	IN	Core value — most common document format
CSV / Excel upload	IN	Proves Python data skills with pandas
Natural language Q&A; with citations	IN	Core differentiator of the product
Document collections (workspaces)	IN	Cross-document queries are the thesis
Streaming responses	IN	UX critical — perception of speed
Export answers as PDF / Markdown	IN	High utility, low complexity

User authentication (Clerk)	IN	Reuse pattern from Mind-Sync
Docx / PowerPoint parsing	IN	python-docx, python-pptx — quick win
Real-time collaboration	OUT	V2 — scoped out for MVP focus
Custom embedding fine-tuning	OUT	V3 — requires training data pipeline
Mobile app	OUT	Web-first; responsive is sufficient for v1
OCR for scanned documents	OUT	V2 — requires Tesseract integration

05 Feature Specifications

FEAT-01 Document Upload & Processing Pipeline

P0 MUST HAVE

Multi-format document ingestion with async processing. Files are chunked, embedded, and stored in a vector database. Users see real-time processing status.

Acceptance Criteria

- ✓ Supports PDF, CSV, XLSX, DOCX, TXT, PPTX formats
- ✓ Files up to 50MB per upload, 10 files per collection
- ✓ Async processing with progress indicator (0% to Parsing to Chunking to Embedding to Ready)
- ✓ Chunking strategy: 512 tokens with 64-token overlap for context preservation
- ✓ Duplicate detection via SHA-256 hash — prevents re-processing
- ✓ Error states shown with actionable messages (corrupted file, unsupported encoding)
- ✓ Processing queue with real-time job status via WebSocket updates

FEAT-02 RAG-Powered Q&A; with Citations

P0 MUST HAVE

Core feature. User asks a natural language question. Aria retrieves the top-k relevant chunks via semantic search, constructs a context-aware prompt, and streams the answer with exact source references.

Acceptance Criteria

- ✓ Semantic search retrieves top-5 most relevant chunks via cosine similarity
- ✓ Every answer includes citations: [Source: filename.pdf, Page 4, Para 2]
- ✓ Streaming response — first token appears within 800ms
- ✓ Confidence indicator per citation (High / Medium / Low based on similarity score)
- ✓ Cross-document synthesis — answers across multiple uploaded files simultaneously
- ✓ Follow-up questions maintain conversation context (last 5 turns)
- ✓ Graceful fallback: 'I could not find relevant information in your documents'

FEAT-03 Document Collections (Workspaces)

P0 MUST HAVE

Users organize documents into named collections. Each collection has its own vector namespace, chat history, and settings — enabling separation between e.g. 'Q3 Research' and 'Legal Docs'.

Acceptance Criteria

- ✓ Create, rename, delete collections
- ✓ Each collection has isolated vector namespace in ChromaDB
- ✓ Documents can only belong to one collection (no cross-collection query in v1)
- ✓ Collection stats: document count, total pages, last queried timestamp
- ✓ Chat history per collection — persistent across sessions

FEAT-04 **Smart Document Summary** P1 SHOULD HAVE

Auto-generate structured summaries for any uploaded document. Uses map-reduce summarization for long documents — each chunk summarized independently, then synthesized into a final summary.

Acceptance Criteria

- ✓ One-click summary generation per document
- ✓ Map-reduce strategy for documents > 10,000 tokens
- ✓ Output: TL;DR (3 sentences), Key Points (5 bullets), Topics Covered, Sentiment
- ✓ Summary generated once and cached — not regenerated on re-open
- ✓ Exportable as Markdown or PDF

FEAT-05 **CSV / Excel Data Analysis** P1 SHOULD HAVE

When a CSV or Excel file is uploaded, Aria detects it and enables data-analysis mode. Users ask questions about the data and get answers computed via pandas — not just retrieved.

Acceptance Criteria

- ✓ Auto-detects CSV/XLSX and switches to data analysis mode
- ✓ Natural language to pandas query generation via Gemini 2.0
- ✓ Executes queries in a sandboxed Python environment (no arbitrary code execution)
- ✓ Returns computed answers: 'Average revenue in Q3 was Rs 42.3L'
- ✓ Auto-generates charts for numerical queries (bar, line, scatter via Plotly)
- ✓ Shows the generated pandas code for full transparency
- ✓ Schema preview on upload: column names, types, row count, null counts

FEAT-06 **Export & Share** P1 SHOULD HAVE

Users can export their Q&A; sessions and document summaries in multiple formats for sharing or archiving purposes.

Acceptance Criteria

- ✓ Export full chat session as Markdown with citations preserved
- ✓ Export as PDF with Aria branding and source references
- ✓ Copy individual answers to clipboard with one click
- ✓ Shareable read-only link to a Q&A; session (72-hour expiry)

FEAT-07 **Source Viewer** P2 NICE TO HAVE

When a citation is clicked, a side panel opens showing the exact source chunk highlighted in context within the original document.

Acceptance Criteria

- ✓ Click any citation — source panel opens on right side
- ✓ Highlights the exact retrieved chunk within surrounding context
- ✓ Shows document name, page number, and character position
- ✓ PDF viewer for PDF sources (react-pdf)
- ✓ Table viewer for CSV/Excel sources

06 User Flows

Primary Flow — First-time user asking a question

01 Land on Homepage
Sees hero demo and value proposition. Clicks 'Try Aria Free' — Clerk sign-up flow begins.

02 Create a Collection
Names the collection (e.g. 'Market Research'). Clicks Create. Empty state shown with upload prompt.

03 Upload Documents
Drags 3 PDFs into upload zone. Processing pipeline starts asynchronously. Progress bar per file — all show 'Ready' state.

04 Ask a Question
Types 'What are the top 3 market risks mentioned across these reports?' in chat input, hits Enter.

05 Receive Cited Answer
Answer streams in within 800ms. 3 risks listed with [Report A, p.12] and [Report C, p.4] citations rendered inline.

06 Explore Source
Clicks citation. Source viewer opens showing highlighted chunk in PDF context with surrounding text.

07 Export
Clicks Export. Downloads full session as Markdown with all citations preserved.

07 Technical Architecture

Frontend	Next.js 15 + React 19 . Tailwind CSS + shadcn/ui . react-pdf + Plotly.js
API Layer	FastAPI (Python 3.11) . WebSocket streaming . Clerk JWT auth middleware
AI / ML Core	LangChain RAG pipeline . Gemini 2.0 Flash (LLM) . Google text-embedding-004
Document Processing	PyMuPDF (PDF parsing) . pandas (CSV/Excel) . python-docx / python-pptx
Storage	ChromaDB (vectors) . PostgreSQL (metadata) . Cloudflare R2 (raw files)

Full Technology Stack Rationale

Layer	Technology	Why This Choice
API Framework	FastAPI	Async-first, automatic OpenAPI docs, fastest Python framework available
RAG Orchestration	LangChain	Industry standard, huge ecosystem, strong retriever abstractions
LLM	Gemini 2.0 Flash	Free tier, fast, consistent with Mind-Sync architecture decision
Embeddings	text-embedding-004	Free, 768-dim, strong semantic search performance
Vector DB	ChromaDB	Local-first, no infra setup, embeds directly in Python process
PDF Parsing	PyMuPDF (fitz)	Fastest Python PDF library, preserves layout and page metadata
Data Analysis	pandas + Plotly	Industry standard, proves Python data skills explicitly
Task Queue	Celery + Redis	Async doc processing without blocking API requests
Auth	Clerk	Reuse from Mind-Sync — consistent authentication pattern
Metadata DB	PostgreSQL + SQLAlchemy	Stores collections, document metadata, chat history

File Storage	`Cloudflare R2`	S3-compatible, generous free tier, cheap egress
Deployment	`Railway + Vercel`	Railway runs Python + Redis + Postgres. Vercel for Next.js

08 RAG Pipeline — Deep Spec

This is the engineering heart of Aria. Every design decision here directly affects answer quality and latency. The pipeline is split into two phases: ingestion (offline, async) and query (online, streaming).

Ingestion Pipeline

```
File upload > R2 storage > Celery task queued > PyMuPDF/pandas parse > RecursiveCharacterTextSplitter (512 tokens, 64 overlap) > Google embedding API (batch size = 100) > ChromaDB upsert with metadata (doc_id, page, char_start, char_end, filename)
```

Query Pipeline

```
User query > embed query > ChromaDB similarity search (top-k=5, threshold=0.75) > filter by collection namespace > retrieve chunks + metadata > construct prompt with citations > Gemini streaming > parse citations > stream to frontend via WebSocket
```

Chunking Strategy

```
512-token chunks with 64-token overlap. Overlap ensures context boundary sentences are never lost. Separator hierarchy: paragraph > sentence > word. Metadata stored per chunk for precise citation generation.
```

Citation Format

```
Model instructed via system prompt to output citations as structured JSON tags: [CITE:doc_id:chunk_id] Frontend parses these inline and replaces with interactive citation badges linked to the source viewer panel.
```

09 Non-Functional Requirements

Answer Latency (p95)	**< 3 seconds**	From query submission to first streamed token. Achieved via async embedding, pre-indexed vectors, and streaming LLM response.
Document Processing	**< 90 seconds**	10-page PDF processed in < 15 seconds. 100-page PDF in < 90 seconds. Async via Celery — never blocks the UI.

Security	**Zero cross-user leakage**	All vectors namespaced by user_id in ChromaDB. Files stored in private R2 bucket with signed URLs. JWT validated on every request.
File Safety	**Multi-layer validation**	File type validated server-side via magic bytes (not just extension). Max 50MB enforced at API gateway. Virus scanning via ClamAV in v2.
Rate Limiting	**Redis sliding window**	10 queries/min per user. 5 uploads/hour per user. Redis-backed sliding window counters prevent abuse.
Sandbox Safety	**RestrictedPython**	CSV pandas code runs in RestrictedPython sandbox — no filesystem access, no network calls, no subprocess. Hard timeout: 10 seconds.

10 Development Roadmap

M1 Core RAG Pipeline

Week 1 (Days 1-7)

Not Started

. FastAPI project setup + PostgreSQL schema definition
. PDF upload endpoint — file to Cloudflare R2 storage
. ChromaDB integration + vector storage with metadata
. RAG Q&A; endpoint with Gemini streaming
. WebSocket processing status updates

M2 Frontend + Auth

Week 2 (Days 8-14)

Not Started

. Next.js 15 setup + Tailwind CSS + shadcn/ui components
. Clerk authentication — sign-up, sign-in, JWT middleware
. Upload zone UI with async progress indicators
. Chat interface with inline citation rendering
. Document collections UI (create / rename / delete)

M3 Data Analysis + Polish

Week 3 (Days 15-21)

Not Started

. CSV/Excel upload + pandas data analysis mode
. Plotly chart generation for numerical queries
. Smart document summary (map-reduce strategy)
. Export as Markdown + Aria-branded PDF
. Full deployment: Railway (backend) + Vercel (frontend)

M4 V2 — Advanced Features

Month 2+

Backlog

. OCR for scanned PDFs using Tesseract
. Cross-collection queries across workspaces
. Team workspaces and sharing
. Fine-tuned embeddings pipeline
. Cohere Rerank for improved retrieval precision

11 Risks & Mitigations

Risk	Impact	Likelihood	Mitigation

Gemini API rate limits hit during load testing	Medium	High	Implement LLM provider abstraction — swap to GPT-4o-mini or Claude Haiku as fallback
ChromaDB data loss on server restart	High	Medium	Use ChromaDB persistent mode with Railway volume mount. Add re-indexing endpoint as recovery
Pandas sandbox escape / code injection	High	Low	RestrictedPython + allowlist of pandas/numpy operations only. Timeout 10s. No exec() calls
Poor answer quality on long documents	Medium	Medium	Tune chunk size and overlap. Add reranking with Cohere Rerank API for better retrieval precision
Railway free tier cold start latency	Medium	High	Keep-alive ping endpoint. Skeleton loading UI. Upgrade to paid tier if demo blocker

12 Portfolio Positioning

Why This Project, Why Now

What It Proves About You

Python backend engineering (FastAPI, async, Celery) . AI/ML pipeline design (RAG, embeddings, vector DBs) . Data engineering (pandas, CSV/Excel) . System architecture across multiple services . Security-aware development (sandboxing, auth, namespacing)

What It Signals to Recruiters

Full-stack with Python proves you are not JS-only . LangChain + ChromaDB shows you understand the AI engineering stack — not just calling GPT-4 . A shipped RAG product is what most AI/ML roles actually do . Data analysis mode shows pandas competence for data engineering roles

How It Complements Mind-Sync

Mind-Sync = TypeScript-heavy, frontend-complex, full-stack. Aria = Python-heavy, AI/ML-focused, backend-complex. Together they cover both sides of the stack and both sides of the market: web product roles AND AI/ML engineering roles.

Industry Relevance

RAG is the #1 pattern in production AI right now. Every company building with LLMs is implementing retrieval-augmented generation. This is not a trend — it is the dominant architecture. Building it from scratch signals you understand it deeply, not just via wrapper APIs.

Your Next Steps

1. Run /gsd-new-project in Antigravity + GSD — paste the problem statement and requirements

2. Let Opus 4.6 generate SPEC.md, ROADMAP.md, and REQUIREMENTS.md from this PRD

3. Execute M1 — FastAPI + PDF upload + ChromaDB + basic Q&A; — working demo in 7 days